# Balanced String — Project Report

## Problem Definition

A string is **balanced** if it contains exactly two different characters and both appear exactly the same number of times. The goal is to find the length of the longest balanced substring of a given string *S*.

### Examples:

| Input | Output | Longest Balanced Substring |
|---|---|---|
| "cabbacc" | 4 | "abba" |
| "abababa" | 6 | "ababab" or "bababa" |
| "aaaaaaa" | 0 | None |

# First Algorithm — Non-Recursive (Iterative)

## Pseudocode

```
Start:
  maxLen = 0

  For every starting point i in
the string:
    Reset freq (all characters =
0)
    diff = 0

    For every character j
starting from i:

      If we see this character
for the first time:
        diff++

      Increment character count
in freq

      If diff > 2:
        Stop and move to next
starting point

      If diff == 2:
        Get count of first
character  → first
        Get count of second
character → second

        If first == second:
          len = j - i + 1
          If len > maxLen:
            maxLen = len

Return maxLen
```

## Implementation (C++)

```
#include <iostream>
#include <string>
#include <algorithm>
#include "Algorithms.h"
using namespace std;
int NonRecursiveAlgorithm(const string& s)
{ int n = s.length(); int maxLen = 0; for
(int i = 0; i < n; i++) { int freq[26] =
{0}; int diff = 0; for (int j = i; j < n;
j++) { int index = s[j] - 'a'; if
(freq[index] == 0) { diff++; }
freq[index]++; if (diff > 2) { break; } if
(diff == 2) { int first = 0; int second =
0; for (int k = 0; k < 26; k++) { if
(freq[k] > 0) { if (first == 0) first =
freq[k]; else second = freq[k]; } } if
(first == second) { int len = j - i + 1; if
(len > maxLen) maxLen = len; }
}
} } return
maxLen;
}
```

## Analysis & Complexity

| Step | Operation | Cost |
|---|---|---|
| 1 | Outer loop iterates over all starting positions | O(n) |
| 2 | Inner loop extends window from each start | O(n) |
| 3 | Frequency update and two-char check (26 slots) | O(1) |
| | Total Time Complexity | $O(n^2)$ |
| | Space Complexity | O(1) |

## Second Algorithm — Recursive

### Pseudocode

```
FUNCTION solve(S, start, index, freq[], diff,
maxLen): IF index == length(S): RETURN c <- S[index]
- 'a' IF freq[c] == 0: diff <- diff + 1 freq[c] <-
freq[c] + 1 IF diff > 2: RETURN IF diff == 2:
first <- 0, second <- 0
FOR i FROM 0 TO 25:
IF freq[i] > 0:
IF first == 0: first <- freq[i]
ELSE: second <- freq[i] IF first == second: maxLen
<- MAX(maxLen, index - start + 1) CALL solve(S,
start, index + 1, freq, diff, maxLen)

FUNCTION RecursiveAlgorithm(S):
maxLen <- 0 FOR i FROM 0 TO
length(S)-1: freq[26] <- {0} CALL
solve(S, i, i, freq, 0, maxLen)
RETURN maxLen
```

## Implementation (C++)

```
#include <iostream>
#include <string>
#include <algorithm>
#include "Algorithms.h"
using namespace std;

void solve(const string& s, int start, int
index, int freq[26], int diff, int& maxLen) { if
(index == s.size()) { return; } int c = s[index]
- 'a'; if (freq[c] == 0) { diff++; } freq[c]++;
if (diff > 2) { return; } if (diff == 2) { int
first = 0, second = 0; for (int i = 0; i < 26;
i++) { if (freq[i] > 0) { if (first == 0) first
= freq[i]; else second = freq[i]; } } if (first
== second) { maxLen = max(maxLen, index - start
+ 1); } } solve(s, start, index + 1, freq, diff,
maxLen); }

int RecursiveAlgorithm(const string& s)
{ int maxLen = 0; for (int i = 0; i <
s.size(); i++) { int freq[26] = {0};
solve(s, i, i, freq, 0, maxLen); }
return maxLen;
}
```

## Analysis & Complexity

| Step | Operation | Cost |
|---|---|---|
| 1 | Outer loop calls solve() for each starting index | O(n) |
| 2 | Each solve() call recurses at most (n - start) levels | O(n) |
| 3 | Each recursion level does constant work | O(1) |
| 4 | Call stack depth per start position | O(n) space |
| | Total Time Complexity | $O(n^2)$ |
| | Space Complexity | O(n) |

## Comparison

| Feature | Iterative (NonRecursiveAlgorithm) | Recursive (RecursiveAlgorithm) |
|---|---|---|
| Time Complexity | $O(n^2)$ | $O(n^2)$ |
| Space Complexity | O(1) | O(n) — call stack |
| Stack Overflow Risk | No | Yes (very long strings) |
| Performance | Faster | Slower (function call overhead) |
| maxLen scope | Local variable | Local variable (passed by reference) |
| String parameter | const string& | const string& |

## Understanding

Both algorithms share the same core idea: fix a starting index, then try to extend the window one character at a time. At each step, we track how many distinct characters exist in the current window (*diff*) and how many times each appears (*freq[]*). The window is abandoned immediately when *diff* exceeds 2.

If the window contains exactly two distinct characters with equal frequency (*first == second*), we have found a balanced substring and update our maximum.

The iterative version uses nested loops and is preferred in practice because it runs in constant extra space and avoids the overhead of recursive calls.

The recursive version (*solve*) is a forward recursion — it does not backtrack. Because *diff* is passed by value and *freq[]* is reset for each starting index in the outer loop, each call chain is independent. The function simply advances the index until the string ends or *diff* exceeds 2, at which point it returns.

In the updated implementation, **maxLen** is no longer a global variable. It is declared locally in *RecursiveAlgorithm* and passed to *solve* by reference (*int& maxLen*), eliminating any risk of cross-call contamination. Similarly, both functions now accept the input string as **const string&** to avoid unnecessary copying.